

svdev

Gestore di ambienti di sviluppo locali multi-tenant

Stefano Videsott · Matricola 252739

Fondamenti di Amministrazione di Sistema

Università degli Studi di Trento · 25 Giugno 2026

codeberg.org/StefanoVidesott/svdev

Il Problema

Gestire più ambienti di sviluppo locali è faticoso.

conflict — porte, nomi di rete e nomi di volume in conflitto tra progetti

scattered — un `docker-compose.yml` diverso in ogni cartella di progetto

verbose — `docker compose -f /percorso/lungo/compose.yml up -d` ogni volta

no standard — ogni progetto inventa le proprie convenzioni per porte, immagini e configurazione

Origine

Prima – shell script

```
# da eseguire con source in ogni nuovo terminale
source ~/dev.sh

# fragile e verboso da scrivere
```

Uno strumento personale da `source`-are in ogni sessione. Usato in contesto aziendale.

Questo progetto

```
# installazione in una riga
curl -fsSL .../install.sh | bash

# poi semplicemente
svdev tenant demo start
```

Riscrittura delle features principali come package Python installabile. Conventional commit, struttura modulare.

svdev

Una CLI Python che gestisce ambienti di sviluppo locali multi-tenant tramite Docker Compose — un'immagine condivisa, stack isolati, zero conflitti di porte.

tenant

Uno stack isolato: container, porta, rete e volume propri. Configurato tramite `.env`, esteso tramite `router.py`.

core

Un'immagine Docker FastAPI condivisa. Lo stesso binario gira per ogni tenant — la logica per-tenant viene iniettata a runtime tramite volume mount.

CLI

Genera file compose da un template, assegna le porte automaticamente e fa da wrapper alle operazioni lifecycle di docker compose.

Come funziona un Tenant

Sorgente — nella repo:

```
tenants/demo/  
├── .env  
└── router.py
```

```
# tenants/demo/.env  
TENANT_NAME=demo  
APP_PORT=9001  
DEBUG=True  
CORE_IMAGE=svdev-core:live
```

```
# tenants/demo/router.py (generato da svdev new)  
from fastapi import APIRouter  
router = APIRouter()  
  
@router.get("/hello")  
def hello():  
    return {"message": "Hello from demo tenant"}
```

Runtime — dentro il container:

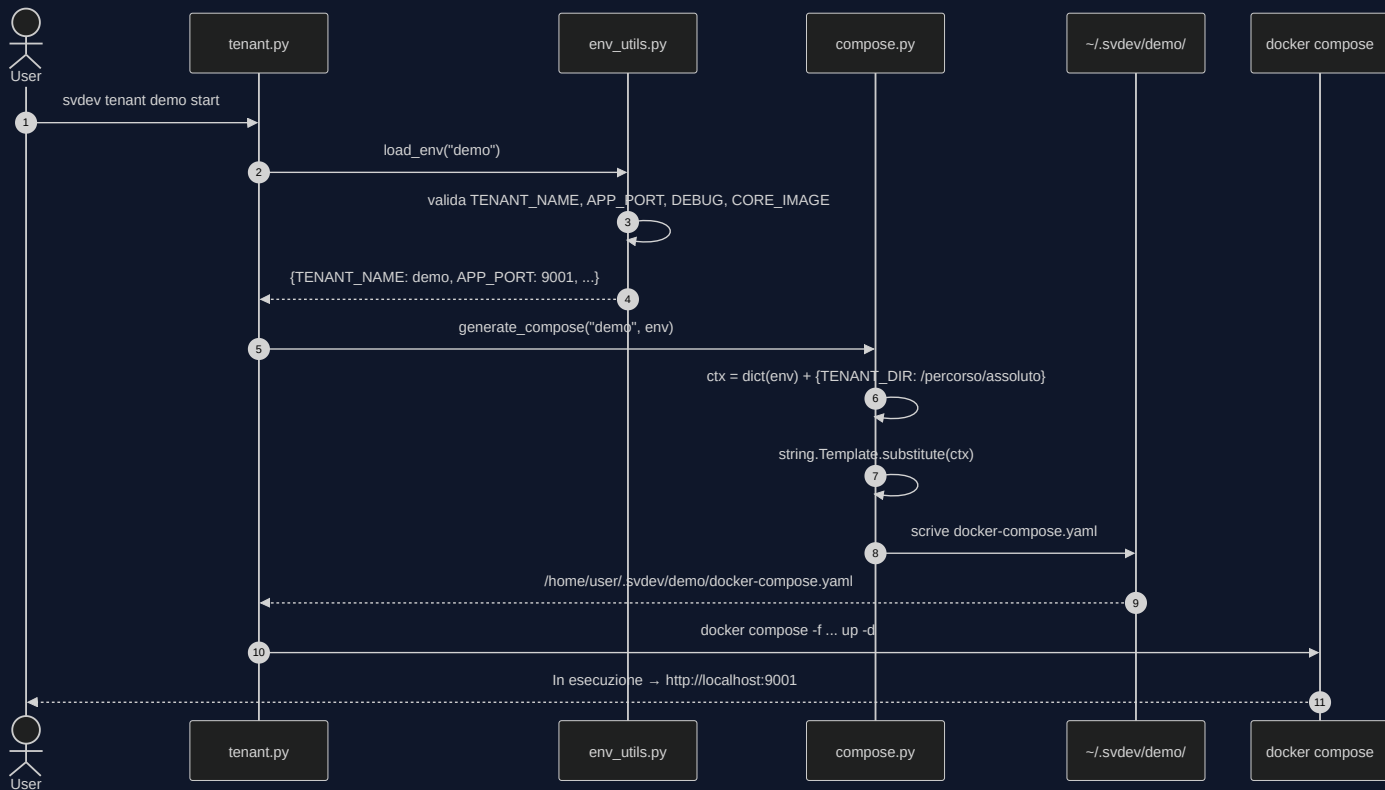
```
# app/main.py - import dinamico all'avvio  
def _load_tenant_router():  
    router_path = Path("/app/tenant/router.py")  
    spec = importlib.util.spec_from_file_location(  
        "tenant_router", router_path  
    )  
    module = importlib.util.module_from_spec(spec)  
    spec.loader.exec_module(module)  
    if hasattr(module, "router"):  
        app.include_router(module.router, prefix="")  
  
_load_tenant_router()
```

```
# uvicorn monitora la cartella tenant → hot reload  
uvicorn.run("main:app", host="0.0.0.0", port=8000,  
            reload=True, reload_dirs=["/app/tenant"])
```

Il volume `${TENANT_DIR}:/app/tenant` mappa la cartella sorgente dentro il container. Modifica `router.py` in locale → uvicorn ricarica istantaneamente.

Flusso di Generazione del Compose

```
svdev tenant demo start
```



Comandi CLI

Globali

Comando	Azione
<code>svdev build [image]</code>	Costruisce l'immagine Docker core
<code>svdev new <nome></code>	Crea un tenant, assegna la porta automaticamente
<code>svdev list</code>	Mostra tutti i tenant con stato/porta/uptime/health

`svdev list` usa il Docker SDK per leggere stato, uptime e immagine di ogni container in esecuzione.

Per-tenant — `svdev tenant <nome> ...`

```
start    # carica .env → renderizza template → compose up
stop     # compose stop
restart  # compose restart
logs     # compose logs -f (Ctrl+C per uscire)
bash     # exec core /bin/bash
status   # pannello Rich: stato, porta, uptime, health, im
edit     # apre .env in $EDITOR (fallback: vim)
remove   # compose down -v (richiede conferma)
```

`status` e `list` usano il Docker SDK per l'ispezione e `requests` per l'health check HTTP su `/health`.

Scelte Tecniche Chiave

1 Editable install (`pip install -e .`)

Ogni modulo usa `Path(__file__).parent.parent.parent` per trovare templates e tenant. Con un'installazione normale `__file__` punterebbe a site-packages.

2 Click invece di argparse — subcommand annidati nativi

`svdev tenant <nome> start` corrisponde a un `@click.group()` con argomento, contenente subcommand `@tenant.command()`. Con argparse servirebbero sub-parser manuali.

3 Ibrido Docker SDK + subprocess

SDK (package `docker`) per interazione con container e subprocess per i Compose.

4 `string.Template` per il rendering del compose

Usa la sintassi `${VAR}` — identica all'interpolazione nativa di Docker Compose. Il template è leggibile senza conoscere Python. I file generati restano fuori dal version control.

Il Tenant docs

La documentazione vive dentro il sistema stesso.

```
# tenants/docs/router.py (estratto)
PAGES = {
    "home":      ("introduction", "Introduction"),
    "motivation": ("motivation",   "Motivation"),
    "architecture": ("architecture", "Architecture"),
    "cli":        ("cli",          "CLI Reference"),
    "installation": ("installation", "Installation"),
    "new-tenant": ("new_tenant",    "Adding a tenant"),
}

@router.get("/", response_class=HTMLResponse)
def home():
    return _render_page("home")
```

Pipeline di rendering

```
def _render_page(page_key: str) → str:
    page_file, title = PAGES[page_key]
    md_text = (PAGES_DIR / f"{page_file}.md").read_text()
    body = markdown.markdown(
        md_text,
        extensions=["fenced_code", "tables", "toc"]
    )
    template = jinja_env.get_template("page.tmpl.html")
    return template.render(body=body, title=title, ...)
```

Usa la propria immagine Docker `stefanovidesott/svdev-core:docs` con `markdown` + `jinja2` pre-installati.

La stessa sorgente Markdown → HTML tramite questo router — dimostra come il modello tenant supporti output diversi da un'unica sorgente.

Showcase — Stessa Infrastruttura, Output Diversi

docs

Documentazione di progetto servita come web app multi-pagina. Sorgente Markdown → HTML tramite `router.py` personalizzato + template Jinja2. Porta 9000.

portfolio

Portfolio personale reale. Clonato con HTTrack — HTML/CSS statici serviti direttamente dal volume mount. Routing multilingua (`/{{lang}}/{{page}}`).

freelance

Sito demo per agenzia freelance. Template HTML Jinja2 renderizzati lato server via router FastAPI.

agency

Sito demo per agenzia creativa. Asset statici + template Jinja2. Tutti e quattro girano simultaneamente su porte diverse.

Tre esperienze web completamente diverse — un'unica immagine Docker `svdev-core` condivisa.

Live Demo

```
svdev list
```

```
svdev tenant docs start
```

```
svdev new live-demo
```

```
svdev tenant live-demo start
```

Sviluppi Futuri

→ Template compose più ricchi — database per-tenant

→ Più servizi per tenant — modello attuale: un container; futuro: più servizi configurabili per ogni tenant

Nuovi comandi = nuovo file in `svdev/commands/`. Nuove funzionalità compose = modifiche al template. Architettura progettata per estendersi senza riscritture.

Grazie per l'attenzione

Stefano Videsott · Matricola 252739

codeberg.org/StefanoVidesott/svdev